Módulo I - Front end estático

Prof. Roosevelt Carvalho Franklin Santos

Aula 05 | CSS Layout & Responsividade

Problema Gerador

O que vamos resolver na aula de hoje?

O Cenário: "Nossa Aumiau Pet Store já tem estrutura semântica (Aula 04). Mas olhem para ela: é uma 'tripa' vertical. Para ver o produto lá de baixo, tenho que rolar a página e perco o menu de vista. Além disso, as imagens dos produtos estão uma embaixo da outra, desperdiçando espaço nas laterais do monitor."

O Problema: "O HTML padrão empilha tudo (bloco). Sites modernos colocam coisas lado a lado (cards de produtos) e mantêm menus visíveis. Como dizer ao navegador: 'Coloque esses 3 produtos na mesma linha' ou 'Mantenha esse menu colado no teto'?"

A Solução: "Poderes avançados do CSS: Flexbox para alinhar coisas, Grid para criar galerias robustas e Position para fixar elementos. E, claro, a Responsividade para que tudo isso se adapte ao celular."

ada

A "Árvore Genealógica" do CSS

Às vezes, queremos estilizar um p, mas apenas se ele estiver dentro de um footer.

Espaço (Descendente): `div p { ... }`

- Seleciona *todos* os parágrafos dentro da div, não importa quão fundo estejam (filhos, netos ...).

Maior que (Filho Direto): `div > p { ... }`

- Seleciona apenas os filhos diretos. Se o p estiver dentro de outra caixa dentro da div, ele é ignorado.

Vírgula (Grupo): `h1, h2, h3 { ... }`

- Aplica o mesmo estilo para todos eles.

ada

Pseudo-classes do CSS (estados)

Selecionam elementos baseados em seu estado, não só no nome.

`:hover`

- Quando o mouse passa por cima (Ex: mudar cor do botão).

`:nth-child(2)`

- Seleciona o segundo filho de uma lista (Ex: pintar linhas da tabela alternadas).

`:not(.destaque)`

- Seleciona tudo o que *não* tem a classe destaque.

A propriedade position

static

- O padrão. O elemento segue a ordem do HTML.

relative

- Permite mover o elemento (top, left) sem afetar os vizinhos.

absolute

- Remove o elemento do fluxo. Ele flutua sobre os outros. Posiciona-se em relação ao pai mais próximo que não seja static. (Ótimo para etiquetas de "Promoção" em cima de fotos).

fixed

- Fixa o elemento na tela do navegador. Mesmo rolando, ele fica lá (Ex: botão de WhatsApp, Navbar fixa).

sticky

- O "híbrido". Ele age como normal até você rolar a página, aí ele "gruda" em uma posição (Ex: Cabeçalhos de listas).

ada

Flexbox (O rei do 1D)

Criado para alinhar itens em uma direção (linha ou coluna).

- No container pai: `display: flex;`
- `flex-direction`: row (linha - padrão) ou column (coluna).
- `justify-content`: Alinha no eixo principal (Ex: center, space-between espalha os itens).
- `align-items`: Alinha no eixo cruzado (Ex: center alinha verticalmente no meio).
- `gap`: Espaço entre os itens.

Para entender melhor e de forma completa eu sugiro o conteúdo desse site aqui:
https://origamid.com/projetos/flexbox-guia-completo/

ada

Grid (O rei do 2D)

Criado para controlar linhas e colunas ao mesmo tempo.

Ideal para galerias de fotos e estruturas completas de páginas.

No container pai: `display: grid;`

`grid-template-columns`: Define quantas colunas.

- Ex: `1fr 1fr 1fr` (cria 3 colunas de tamanhos iguais).
- *fr = fração do espaço disponível*

No link abaixo você encontra um material completo para entender o grid:
https://www.origamid.com/projetos/css-grid-layout-guia-completo/

ada

Responsividade (Um site, várias telas)

- O site deve funcionar no celular (300px) e na TV (4000px).
- Usamos Media Queries (@media) para aplicar CSS apenas em certas condições.

Conceito Mobile First: Criamos o CSS básico para celular primeiro, e usamos media queries para telas maiores.

Exemplo de código de um CSS mobile-first com media query para dispositivos maiores

```
/* Padrão (Celular) */
.container { flex-direction: column; }

/* Apenas se a tela for maior que 768px (Tablet/PC) */
@media (min-width: 768px) {
   .container { flex-direction: row; }
}
```

ada

Atividade prática guiada

Objetivo: Pegar o código das aulas anteriores e aplicar responsividade, opções de position, flexbox e grid.

Passo a passo no código:

- Position: Colocar um position sticky na header.
- Display flex: Alinhar os produtos com display flex.
- Responsividade e Grid: Adaptar o layout da homepage para celular e testar o funcionamento com display grid.

ada

Exercício para os alunos

Os clientes da Aumiau querem ver fotos dos clientes felizes usando as roupinhas. Vamos criar a seção 'Galeria da Fama' usando CSS Grid.

Requisitos:

1. No arquivo index.html, crie uma nova `<section id="galeria">` antes do footer.
2. Adicione um título `<h2>Clientes Felizes</h2>`.
3. Crie uma `div` com classe `galeria-grid`.
4. Dentro dela, coloque 6 tags `<img>` (use imagens de placeholder de gatos/cachorros).
5. No CSS (O Desafio):
    - Use `display: grid` na `.galeria-grid`.
    - Configure para ter 3 colunas de tamanho igual `(1fr 1fr 1fr)`.
    - Adicione um `gap` de `10px` entre as fotos.
    - Bônus de Responsividade: Crie uma `@media query` para que, em telas menores que `600px`, a galeria mude para 2 colunas (ou 1 coluna).

Dica para os alunos: Lembrem-se que `grid-template-columns: repeat(3, 1fr);` é um atalho para escrever `1fr` três vezes.

ada

Obrigado